Jenkins

✅ What is Jenkins?

Jenkins is an **open-source automation server** used to **automate parts of the software development process**, especially **continuous integration (CI)** and **continuous delivery (CD)**.

Or

Jenkins is an open-source automation tool used for Continuous Integration/Continuous Delivery (CI/CD) of projects.

✅ What is Continuous Integration (CI)?

- CI is a **development practice** where developers frequently merge code into a shared repository.
- Each merge triggers an **automated build and test**, ensuring integration issues are detected early.

✅ Why Jenkins for CI?

- Jenkins **automates the build, test, and deployment** process.
- It **integrates with version control systems (like Git)** and **triggers jobs** on code changes.
- Supports **plugins** to integrate with various tools like Maven, Docker, JIRA, etc.

✅ How to Automate Workflow in Jenkins?

- **Install Jenkins**.
- **Connect to Git** or any version control.
- Create a **Pipeline (scripted or declarative)** or **Freestyle Job**.
- Define steps: build → test → deploy.
- Set **triggers** (e.g., Git push, schedule).
- Run jobs automatically.

✅ Why Jenkins is Required – Before vs. After:

Aspect	Before Jenkins	After Jenkins
Build Process	Manual, error-prone	Automated, consistent
Code Integration	Infrequent, high-risk merges	Frequent, low-risk merges (CI)
Testing	Manual or delayed	Automated with every change
Feedback Loop	Slow	Fast, real-time
Deployment	Manual	Can be automated (CD)

Akash Kumar

✅ Advantages of Jenkins

- Free and open-source.
- Huge plugin ecosystem.
- Active community.
- Platform-independent (Java-based).
- Scalable with master-agent architecture.

✅ Disadvantages / Drawbacks

- Complex setup for large-scale systems.
- UI can be outdated/cluttered.
- Managing many plugins can cause compatibility issues.
- Requires maintenance (updates, job config, etc.).

✅ Summary

Jenkins is a widely-used CI/CD tool that automates the build, test, and deployment stages of software.
It helps in early bug detection, faster releases, and improved developer productivity. It's essential in modern DevOps workflows.

✅ Jenkins Architecture

- **Master**: Central server that schedules jobs, monitors agents, and provides the UI.
- **Agents (Slaves)**: Execute the jobs assigned by the master.
- **Jobs**: Tasks like build, test, deploy.
- **Plugins**: Extend Jenkins functionalities.

```
[Developer] → [GitHub Push] → [Jenkins Master]
                                    ↓
                            Assigns Job to Agent
                                    ↓
                            [Build/Test/Deploy]
                                    ↓
                            [Feedback to Developer]
```

✅ What is CD (Continuous Delivery / Deployment)?

Continuous Delivery (CD): Automates the delivery of code to **staging/test environments** after CI. Manual approval is often needed before production.

Continuous Deployment: Extends Continuous Delivery to **automatically deploy to production** without manual intervention.

✅ Use-Case & Why CD is Required

Problem Before CD	Solution with CD
Manual deployment was slow & risky	Automated deployment reduces errors & downtime
Delay in release cycles	Faster, frequent, and reliable releases
Hard to replicate environments	Uses scripted pipelines, containers, IaC tools

✅ How to Configure CI/CD in Jenkins

🔹 1. Prerequisites:
- Jenkins installed.
- Git repo with code.
- Build tool (Maven/Gradle).
- Deployment tools: Docker, SSH, Kubernetes, or others.

🔹 2. CI/CD Steps Overview:

1. CI Phase:
   - Clone repo
   - Build code (Maven)
   - Run unit tests
2. CD Phase:
   - Package app (JAR/Docker)
   - Deploy to server/container

✅ What is a Jenkins Pipeline?

A pipeline is a suite of steps that defines the CI/CD workflow in code, often written in a `Jenkinsfile`.

✅What are the key features of Jenkins?

- CI/CD automation
- Plugin support
- Distributed builds (master-agent)
- Pipeline as code (Jenkinsfile)
- Easy integration with Git, Maven, Docker, etc

✅ What is the difference between Declarative and Scripted Pipeline?

- **Declarative**: Simpler, structured syntax (pipeline {} block).
- **Scripted**: Groovy-based, more flexible (node {} block).

✅ What are Jenkins Agents/Nodes?

Agents are machines (virtual or physical) where Jenkins runs jobs. Controlled by the Jenkins master.

Akash Kumar

✅ Sample Jenkins Pipeline Script (Declarative) - groovy script

```
pipeline {
    agent any

    environment {
        APP_NAME = "my-spring-app"
        DOCKER_IMAGE = "myrepo/my-spring-app"
    }

    stages {
        stage('Checkout') {
            steps {
                git 'https://github.com/your-org/your-repo.git'
            }
        }

        stage('Build') {
            steps {
                sh 'mvn clean install'
            }
        }

        stage('Unit Test') {
            steps {
                sh 'mvn test'
            }
        }

        stage('Package Docker Image') {
            steps {
                script {
                    sh "docker build -t ${DOCKER_IMAGE}:${BUILD_NUMBER} ."
                }
            }
        }

        stage('Push Docker Image') {
            steps {
                withCredentials([usernamePassword(credentialsId: 'docker-cred', usernameVariable: 'USER', passwordVariable: 'PASS')]) {
                    sh "echo $PASS | docker login -u $USER --password-stdin"
                    sh "docker push ${DOCKER_IMAGE}:${BUILD_NUMBER}"
                }
            }
        }

        stage('Deploy to Staging') {
            steps {
                sshagent(['staging-server-key']) {
                    sh 'ssh user@staging-server "docker pull ${DOCKER_IMAGE}:${BUILD_NUMBER} && docker restart my-spring-app"'
                }
            }
        }
    }

    post {
        always {
            echo 'Pipeline finished.'
        }
    }
}
```

✅ How to Create Jenkins Pipeline:

- Open Jenkins → New Item → Select **Pipeline**.
- In the Pipeline **section**, paste the above script (Declarative Pipeline).
- Save and **Run**.

✅ Summary

- **CI/CD** automates everything from code commit to production.
- **CI** focuses on build/test.
- **CD** automates delivery/deployment.
- Jenkins pipeline scripts define full workflows.
- You can deploy to VMs, Docker, or Kubernetes depending on setup.

✅ What is a Jenkinsfile?

A `Jenkinsfile` is a **text file** that contains the **pipeline-as-code** for Jenkins. It defines **build, test, deploy** stages of your CI/CD pipeline.
OR
A `Jenkinsfile` is a text file that defines the pipeline script and is placed in the **root directory of the project repository**.

✅ Use Case of Jenkinsfile

- Automate the entire CI/CD flow **within the repo**.
- Version-controlled with your code (Git).
- Allows teams to standardize builds and deployments.

✅ Why is it Required?

Problem	Jenkinsfile Solves
Manual job configuration	Code-based pipeline automation
No version control on jobs	Pipeline is in repo, under Git
Inconsistent build steps	Standardized CI/CD flow

✅ How Jenkins Detects the Jenkinsfile?

When creating a Multibranch Pipeline or GitHub Integration, Jenkins automatically looks for Jenkinsfile at the repo root.

✅ Where to Place Jenkinsfile?

At the **root** of your Git project:

```
/your-spring-boot-app
  ├── src/
  ├── pom.xml
  └── Jenkinsfile
```

✅ How do you trigger a Jenkins job automatically?

- Webhook from Git (on push/PR)
- Cron (Poll SCM)
- Upstream/downstream triggers

✅ What is a plugin in Jenkins?

Plugins extend Jenkins' core functionality, e.g., Git, Maven, Docker, Slack, SonarQube, etc.

✅ 12. What is the Blue Ocean in Jenkins?

A modern UI for Jenkins pipelines with visualizations of stages, branches, and runs.

✅ Basic Jenkinsfile Example (CI + CD)

```
pipeline {
    agent any

    stages {
        stage('Build') {
            steps {
                sh 'mvn clean install'
            }
        }

        stage('Test') {
            steps {
                sh 'mvn test'
            }
        }

        stage('Docker Build & Push') {
            steps {
                script {
                    sh 'docker build -t myapp:latest .'
                    sh 'docker tag myapp:latest myrepo/myapp:latest'
                    sh 'docker push myrepo/myapp:latest'
                }
            }
        }

        stage('Deploy') {
            steps {
                sshagent(['server-key']) {
                    sh 'ssh user@host "docker pull myrepo/myapp:latest && docker run -d --rm -p 8080:8080 myrepo/myapp:latest"'
                }
            }
        }
    }
}
```

✅ Configuration for Jenkinsfile in Project

1. Use a Git repo (GitHub, GitLab, Bitbucket)
2. In Jenkins:
   - Create **Multibranch Pipeline** or **Pipeline job**
   - Point to your Git repo
3. Jenkins will:
   - Auto-detect the `Jenkinsfile`
   - Run pipeline as per the stages

✅ Jenkins Pipeline Architecture (Code + Flow)

```
[Git Repo with Jenkinsfile]
          ↓
[Jenkins Master] ← Webhook from Git (GitHub)
          ↓
[Pipeline Job]
   └─ Stages:
        ├─ Build (Maven)
        ├─ Test (JUnit)
        ├─ Package (Docker)
        └─ Deploy (Remote server / container)
```

✅ Summary

- Jenkinsfile = CI/CD pipeline script inside your repo.
- Jenkins auto-detects it during job setup.
- Enables automated, standardized, version-controlled pipelines.
- Crucial for modern DevOps workflows.

✅ How to write parallel stages in a jenkinsfile(declarative syntax)

```
pipeline {
    agent any

    stages {
        stage('Build') {
            steps {
                echo 'Building the project...'
                sh 'mvn clean install'
            }
        }

        stage('Run Tests in Parallel') {
            parallel {
                stage('Unit Tests') {
                    steps {
                        echo 'Running unit tests...'
                        sh 'mvn test'
                    }
                }

                stage('Integration Tests') {
                    steps {
                        echo 'Running integration tests...'
                        sh './run-integration-tests.sh'
                    }
                }

                stage('Code Quality') {
                    steps {
                        echo 'Running static code analysis...'
                        sh './run-sonarqube.sh'
                    }
                }
            }
        }

        stage('Deploy') {
            steps {
                echo 'Deploying the app...'
                sh './deploy.sh'
            }
        }
    }
}
```

✅ How do you integrate Jenkins with Git?

Use the Git plugin → Configure repository URL and credentials → Set SCM trigger or webhook.

✅ What is the difference between Freestyle Project and Pipeline?

- Freestyle: GUI-based configuration.
- Pipeline: Code-based, more flexible and version-controlled.

✅ What Happens Here?

- Build stage runs first.
- Then 3 stages run in parallel under Run Tests in Parallel:
  - Unit Tests
  - Integration Tests
  - Code Quality
- **Finally, deployment runs.**

✅ When to Use Parallel Stages

- Multiple independent tasks (e.g., unit tests, integration tests, static analysis).
- Different platforms/OS testing.
- Speeds up pipelines and reduces waiting time.

✅ Summary

- Use parallel { ... } inside a stage.
- Each sub-stage inside parallel runs **concurrently**.
- Great for **test execution**, **analysis**, and **platform builds**.

✅ How do you secure Jenkins?

- Use authentication and role-based access
- Use HTTPS
- Limit public access
- Keep plugins and Jenkins updated

✅ How to run stages in parallel in Jenkins?

Use parallel block inside a stage:

```
parallel {
  stage('Test A') { steps { ... } }
  stage('Test B') { steps { ... } }
}
```

✅ How do you handle failed builds?

- Use the post { failure { ... }} block in the pipeline.
- Notify via email or Slack.
- Use retry mechanisms or fix root cause.

✅ Where Is Quality Gate Used?

- Integrated in **CI/CD pipelines** (usually Jenkins + SonarQube).

Akash Kumar

- Runs after code analysis during the **build/test** phase.

✅ What are post actions in Jenkins Pipeline?

They define actions after the pipeline: success, failure, always, unstable, etc.

```
post {
  always { echo 'Runs always' }
  failure { echo 'Runs on failure' }
}
```

Akash Kumar